

```

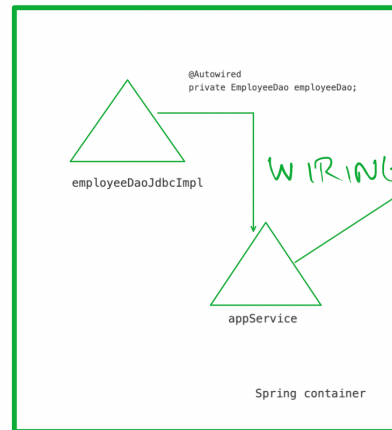
public interface EmployeeDao {
    void addEmployee(Employee e);
}

@Repository
public class EmployeeDaoJdbcImpl implements EmployeeDao {
    ...
}

@Service
public class AppService {
    @Autowired
    private EmployeeDao employeeDao; ✓

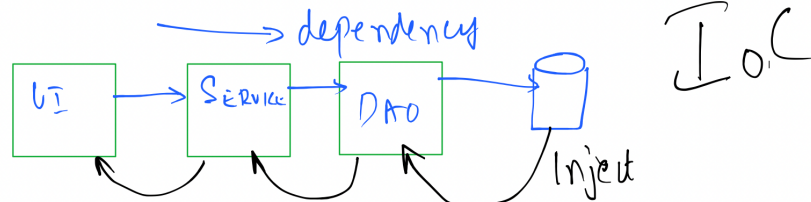
    public void task(Employee e) {
        employeeDao.addEmployee(e);
    }
}

```



READY  
TO  
USE

WIRING



```

@Repository
public class EmployeeRepoJdbcImpl implements EmployeeRepo {
    @Override
    public void addEmployee() {
        System.out.println("Stored in database!!!");
    }
}

```

```

@Service
public class AppService {
    @Autowired
    private EmployeeRepo employeeRepo;

    public void doTask() {
        employeeRepo.addEmployee();
    }
}

```

```

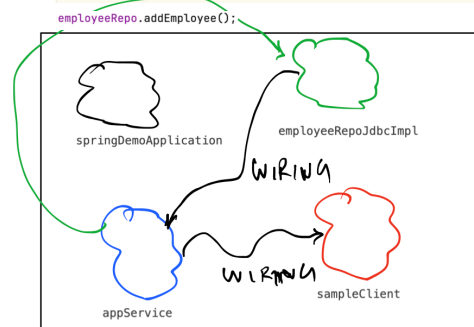
@Component
public class SampleClient implements CommandLineRunner {
    @Autowired
    private AppService service;
    // executes as soon as spring container is created and initialized
    @Override
    public void run(String... args) throws Exception {
        service.doTask();
    }
}

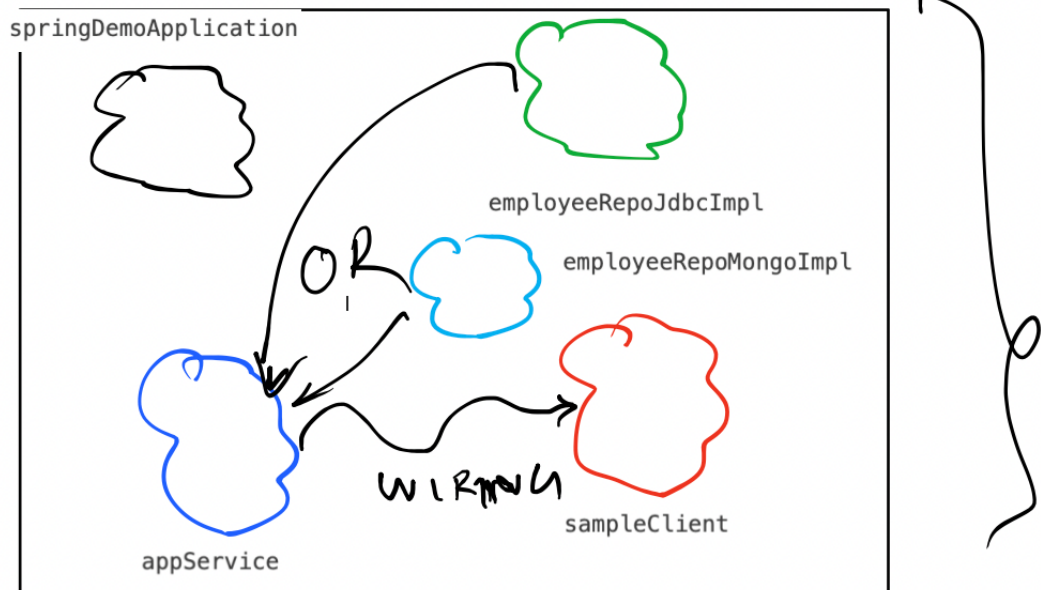
```

```

@SpringBootApplication
public class SpringDemoApplication {
    public static void main(String[] args) {
        // this line is what creates Spring Container
        SpringApplication.run(SpringDemoApplication.class, args);
    }
}

```





Field employeeRepo in AppService required a single bean, but 2 were found:

- employeeRepoJdbcImpl:
- employeeRepoMongoImpl:

Solution:

```
@Service
public class AppService {
    @Autowired
    @Qualifier("employeeRepoJdbcImpl")
    private EmployeeRepo employeeRepo;
```

JDBC

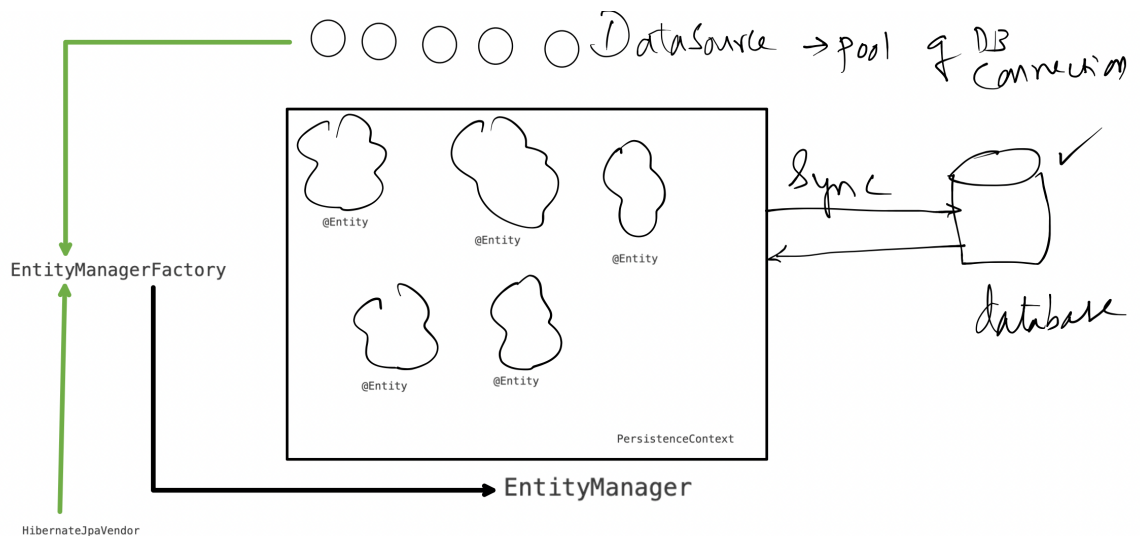
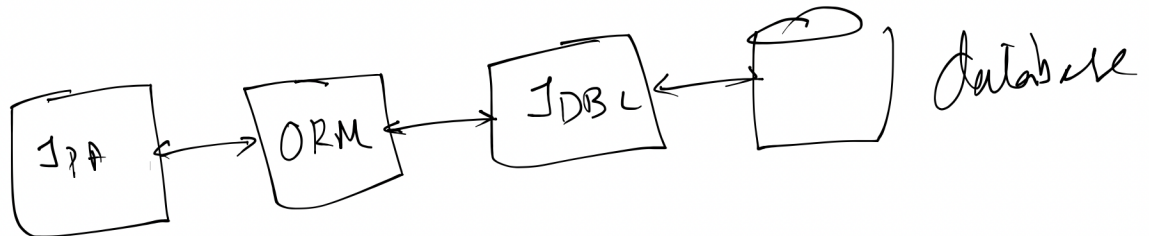
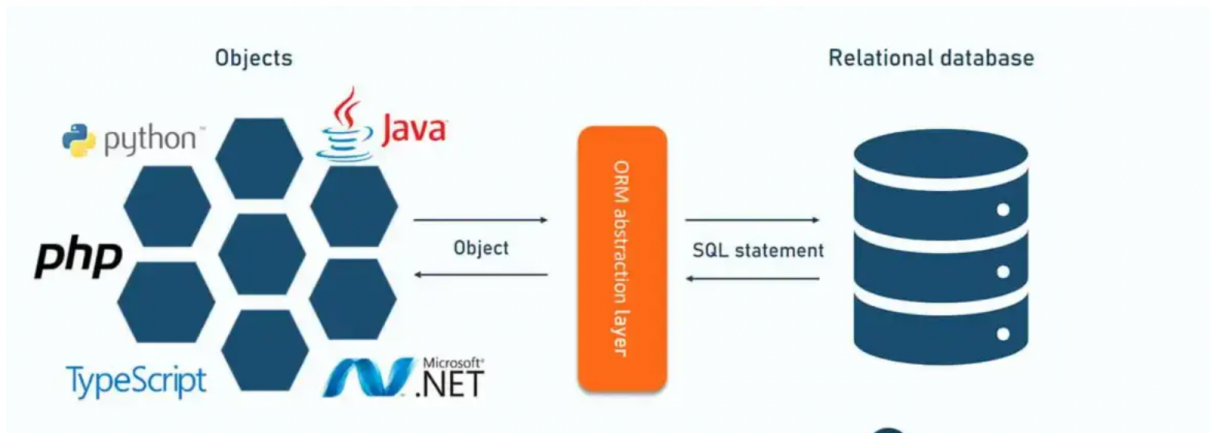
```
@Override
public void addProduct(Product product) throws PersistenceException {
    Connection con = null;
    String SQL = "INSERT INTO products(id, name, price) VALUES (0, ?, ?)";
    try {
        con = DriverManager.getConnection(URL, USER, PWD);
        PreparedStatement statement = con.prepareStatement(SQL);
        statement.setString(1, product.getName());
        statement.setDouble(2, product.getPrice());
        statement.executeUpdate(); // INSERT, DELETE or UPDATE
    } catch (SQLException ex) {
        if (ex.getErrorCode() == 1062) {
            throw new PersistenceException("unable to add product " + product.getId() + " already exists!!", ex);
        } else if (ex.getErrorCode() == 1054) {
            throw new PersistenceException("unable to add product, Bad Input !!!", ex);
        } else {
            throw new PersistenceException("unable to add product " + product + " !!!", ex);
        }
    }
    finally {
        if (con != null) {
            try {
                con.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

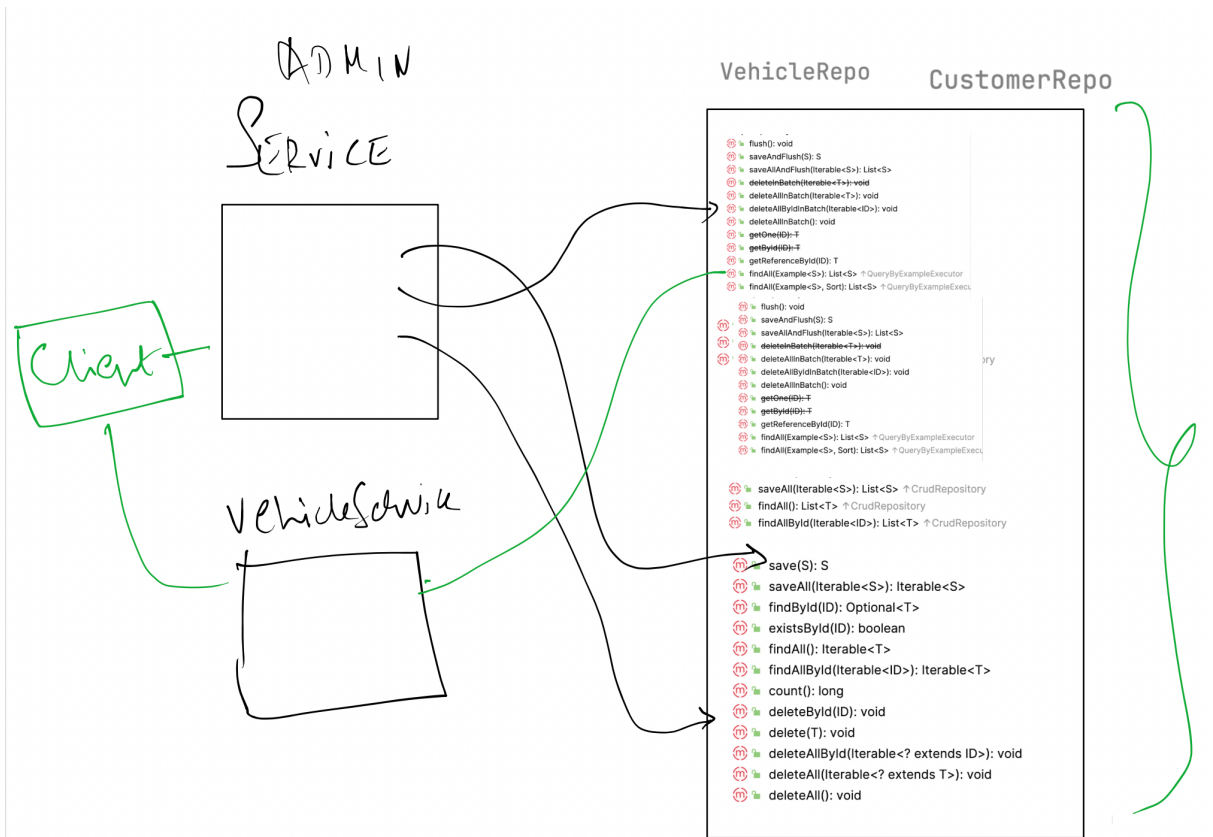
ORM

```
@Override
public void addProduct(Product product) {
    em.persist(product);
}

@Entity
@Table(name="products")
public class Product {
    @Id
    private int id;
    private String name;
    private double price;
}
```

<https://docs.oracle.com/javase/8/docs/api/java/sql/Types.html>





## Build, Execution, Deployment › Compiler › Annotation Processors

+ - ->  
 Default  
 Annotation profile  
 vehicle-re

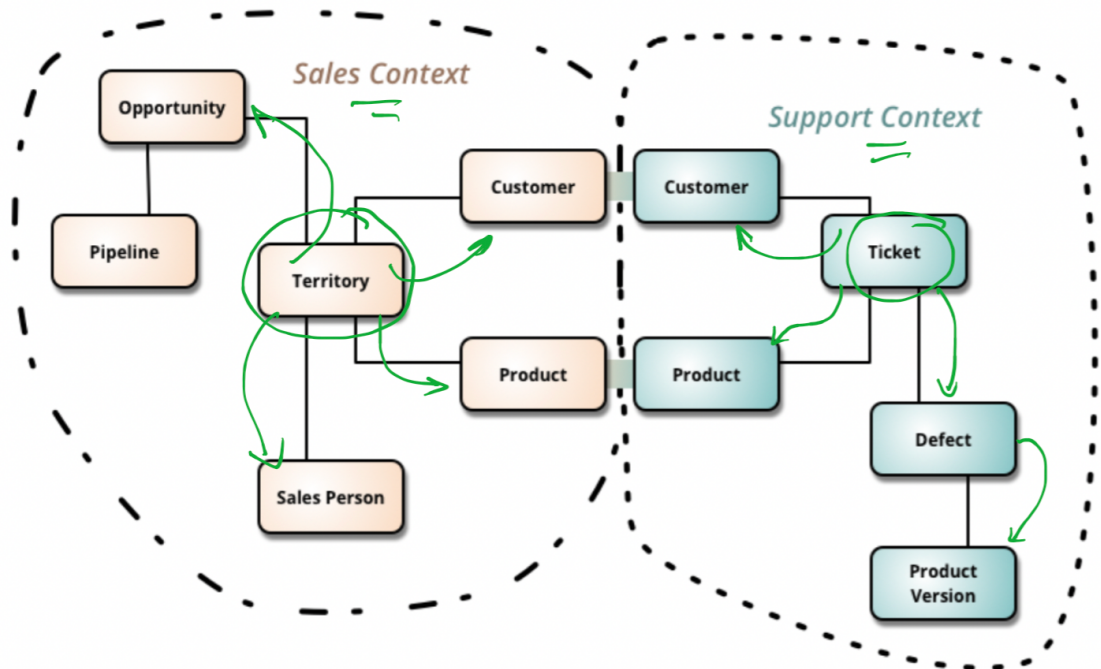
- ☒ Enable annotation processing
- ☒ Obtain processors from project classpath
- ☐ Processor path:

| email           | fname | lname    |
|-----------------|-------|----------|
| anne@adobe.com  | Anne  | Hathaway |
| roger@adobe.com | Roger | Smith    |

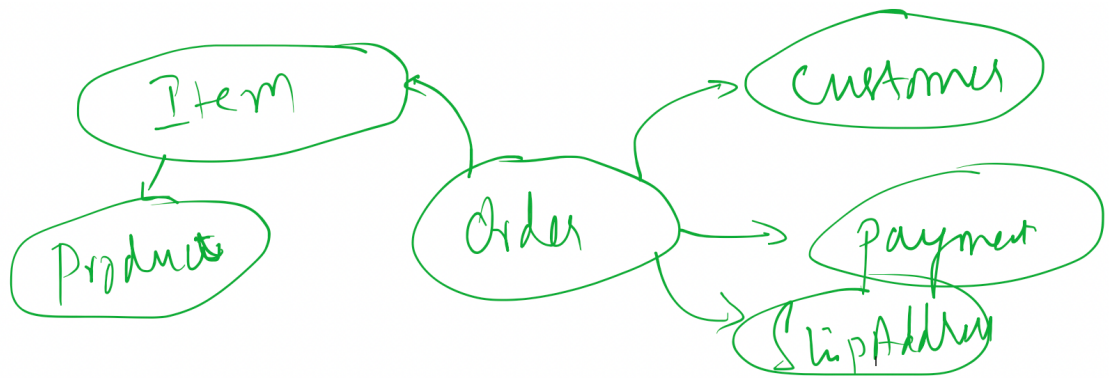
| reg_no        | hire_rate | fuel_type |
|---------------|-----------|-----------|
| DH-10-AA-0434 | 5300      | PETROL    |
| KA-05-AB-1234 | 4500      | PETROL    |
| UP-15-EB-4321 | 6500      | ELECTRIC  |

bookings

| id | customer_fk     | vehicle_fk    | date_from   | date_to     | amount  |
|----|-----------------|---------------|-------------|-------------|---------|
| 23 | anne@adobe.com  | UP-15-EB-4321 | 18-AUG-2025 | null        | 0       |
| 24 | roger@adobe.com | KA-05-AB-1234 | 18-AUG-2025 | 20-AUG-2025 | 8900.00 |
| 25 | roger@adobe.com | DH-10-AA-0434 | 21-AUG-2025 | null        | 0       |
| 26 | peter@adobe.com | KA-05-AB-1234 | 21-AUG-2025 | null        | 0       |







```
mysql> select * from customers;
```

| email           | fname | lname |
|-----------------|-------|-------|
| danny@adobe.com | Danny | Smith |
| rita@adobe.com  | Rita  | Smith |

```
mysql> select * from products;
```

| id | name        | price  | quantity |
|----|-------------|--------|----------|
| 1  | iPhone 16   | 89000  | 98       |
| 2  | Sony Bravia | 289000 | 99       |
| 3  | LG AC       | 54000  | 100      |

```
mysql> select * from orders;
```

| oid | order_date                 | total  | customer_fk    |
|-----|----------------------------|--------|----------------|
| 1   | 2025-08-21 16:58:05.746000 | 467000 | rita@adobe.com |

```
mysql> select * from items;
```

| item_id | amount | qty | product_fk | order_fk |
|---------|--------|-----|------------|----------|
| 1       | 289000 | 1   | 2          | 1        |
| 2       | 178000 | 2   | 1          | 1        |